

Fil Rouge – Optimisation de tournées de véhicules

Méta-heuristiques, Systèmes Multi-Agents et Q-Learning

Adrien Benaroch, Baptiste Neveux, Bila Miali, Lilya Sadoun, Ugo Roccamatysi, Guillaume Darlot

4 mai 2025

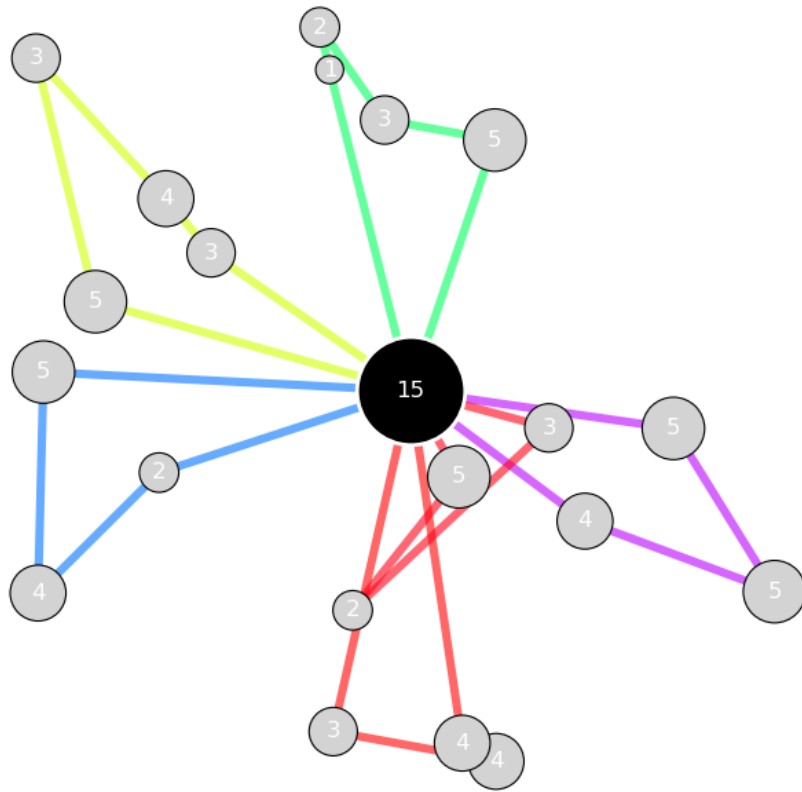


FIGURE 1 – Exemple de solution VRPTW : chaque couleur correspond à la tournée d’un véhicule ; le nombre au centre de chaque nœud indique la demande (entrepôt en noir).

Résumé

Ce projet, réalisé dans le cadre de l’électif *Intelligence collaborative* (ICO), étudie différentes façons de résoudre un problème de tournées de véhicules (VRP). Nous avons testé trois méta-heuristiques classiques (Tabu, Recuit simulé, Algorithme génétique), puis nous les avons combinées avec un système multi-agents et du Q-Learning. Une démo Streamlit permet de lancer les algos, de voir les routes et de comparer les coûts : (cf readme)

Table des matières

1	Introduction	4
1.1	Problem Definition	4
1.2	Computational Complexity	4
2	Environnement expérimental	4
2.1	Approche générale du projet	4
2.2	Génération des instances synthétiques	5
2.3	Modèle de données	6
2.4	Paramètres utilisés dans l'étude	7
2.5	Génération de l'environnement dans l'application	7
3	Architecture logicielle du projet	8
3.1	Vue globale du dépôt Meta/	8
3.2	Interfaces clés.	9
3.3	Flux d'exécution d'une expérience	9
4	Méthodes de résolution Méta-heuristiques	10
4.1	Utilisation d'une méta-heuristique dans l'application	10
4.2	Tabu Search	13
4.2.1	VRPTW-Specific Algorithms	13
4.2.2	Small-Sized Benchmark (10 clients)	14
4.2.3	Large-Sized Benchmark (30 clients)	16
4.3	Simulated Annealing	17
4.3.1	5.2.1 VRPTW-Specific Algorithms	17
4.4	Genetic Algorithms	18
4.4.1	5.3.1 VRPTW-Specific Algorithms	18
4.4.2	Small-Sized Benchmark (40 clients)	19
4.4.3	Large-Sized Benchmark (100 clients)	22
5	Optimization Collaborative	23
5.1	SMA : approche multi-agent	23
5.1.1	Architecture logicielle	24

5.1.2	Paramètres expérimentaux	24
5.2	SMA dans l'application	25
5.2.1	Small-Sized Benchmark (40 clients)	26
5.2.2	Large-Sized Benchmark (100 clients)	27
5.3	Q-Learning	28
5.3.1	Encodage de l'état	28
5.3.2	Ensemble d'actions	29
5.3.3	De la version naïve à la version « best-improvement »	29
5.3.4	Boucle d'apprentissage	30
5.4	Q-Learning en direct dans l'application	31
5.4.1	Small-Sized Benchmark (40 clients)	33
5.4.2	Large-Sized Benchmark (100 clients)	33
6	Global Comparison and Conclusion	35
6.1	Hyper-parameter Summary	36
6.2	Perspectives d'amélioration du protocole.	38
6.3	Limites observées et pistes de recherche.	38

1 Introduction

1.1 Problem Definition

On dispose d'un dépôt (noeud 0) et d'un ensemble de clients $\mathcal{C} = \{1, \dots, N\}$. Chaque client i a une demande q_i et chaque véhicule a une capacité $Q_{\max}$. Une tournée part toujours du dépôt, visite un sous-ensemble de clients, puis revient au dépôt. Le coût d'une solution est

$$C = N_v W + \sum_{(i,j) \in \text{arcs}} c_{ij},$$

où N_v est le nombre de véhicules utilisés, W un poids fixe (nous prenons $W = 100$) et c_{ij} le temps de trajet ou la distance entre les noeuds i et j .

Simplification. Le problème complet VRPTW inclut des fenêtres de temps $[e_i, \ell_i]$ pour chaque client. Pour garder le projet faisable dans le semestre nous **n'avons pas pris ces fenêtres en compte** dans nos algorithmes : seule la capacité des véhicules est respectée. Les données e_i et ℓ_i restent présentes dans les fichiers pour de futurs tests, mais elles ne pèsent pas dans C .

1.2 Computational Complexity

Même sans fenêtres de temps, le VRP est NP-difficile : le cas spécial « un seul véhicule » se ramène au Problème du Voyageur de Commerce (TSP). Ajouter des capacités transforme la recherche locale en un espace de solutions encore plus grand, d'où le recours à des heuristiques et méta-heuristiques. Les fenêtres de temps rendent le problème au moins aussi difficile ; c'est pourquoi beaucoup d'études se contentent de construire de « bonnes » solutions sans garantie d'optimalité, ce que nous faisons ici.

2 Environnement expérimental

2.1 Approche générale du projet

Le dépôt **Meta/** est découpé en trois blocs :

1. **environment/** : création d'instances VRP synthétiques et classes Pydantic qui décrivent les données (`EnvHyperParams`, `CustomerModel`, `VRPTWEnvironment`).
2. **scheduling/** : implémentation des planificateurs (Tabu, Recuit, GA) avec hyper-paramètres typés.

3. **visualization/** : petite appli Streamlit pour piloter tout ça (`ui_utils.draw_routes`, pages multipages, etc.).

Chaque algorithme reçoit donc le même objet `VRPTWEnvironment` ; cela garantit que les résultats sont comparables et que l'on peut changer la taille ou la densité de l'instance sans toucher au code des heuristiques.

2.2 Génération des instances synthétiques

Nous avons choisi de générer nos propres instances pour deux raisons : avoir un contrôle total sur la taille et pouvoir lancer beaucoup d'essais rapides sans dépendre d'un jeu de données externe.

1. **Positionnement des clients.** Les coordonnées sont tirées uniformément dans le carré $[\text{coord_min}, \text{coord_max}]^2$ (par défaut $[-200, 200]$). Le dépôt est placé au centre du carré, ce qui évite des distances extrêmes.
2. **Demandes.** La demande q_i d'un client est un entier uniforme dans $[\text{min_demand}, \text{max_demand}]$.
3. **Matrice de distances.** On calcule la matrice euclidienne complète avec `scipy.spatial.distance.cdist` et on force la diagonale à zéro. Cette même matrice sert de *time matrix*.
4. **Fenêtres de temps (optionnel).** Les bornes e_i et ℓ_i sont stockées mais *pas* utilisées dans le coût,
5. **Graine aléatoire.** Tous ces tirages sont reproductibles grâce au paramètre `random_seed`.

L'Algorithme 2 résume le processus.

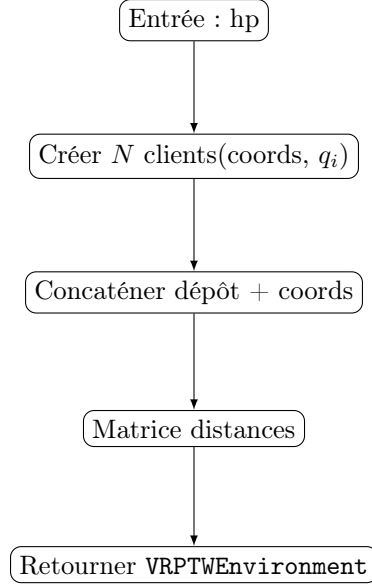


FIGURE 2 – Étapes de `generate_env()`.

2.3 Modèle de données

Le coeur de l’environnement est décrit avec Pydantic :

- `CustomerModel` : $\langle \text{id}, e_i, \ell_i, \text{durée service } s_i, \text{demande } q_i \rangle$.
- `EnvironmentModel` : liste de clients, capacité véhicule, coordonnées, matrice temps.
- `EnvHyperParams` (liste 3) : hyper-paramètres nécessaires à la génération.
- `VRPTWEnvironment` : objet runtime utilisé par les algorithmes (contient la matrice sous forme `pandas.DataFrame`).

```

class EnvHyperParams(BaseModel):
    num_customers: int = 40
    vehicle_capacity: int = 15
    earliest_arrival: float = 0.0
    latest_arrival: float = 1000.0
    service_duration: float = 5.0
    min_demand: int = 1
    max_demand: int = 5
    random_seed: int | None = 42
    coord_min: int = -200
    coord_max: int = 200
  
```

FIGURE 3 – Hyper-paramètres de l’environnement.

2.4 Paramètres utilisés dans l'étude

Sauf mention contraire, toutes les expériences emploient les valeurs du Tableau 1. Nous jouons parfois sur `num_customers` pour passer de « small » (40 clients) à « large » (100 clients).

Paramètre	Symb.	Valeur par défaut
Nombre de clients	N	40
Capacité véhicule	$Q_{\max}$	15
Demande min / max	$q_{\min}, q_{\max}$	1 – 5
Carré des coordonnées	—	$[-200, 200]^2$
Service time	s_i	5 min
Fenêtres e_i, ℓ_i	—	0 – 1000 (ignoré)
Pénalité véhicule	W	100
Graine aléatoire	—	42

TABLE 1 – Valeurs de référence utilisées dans les benchmarks.

Ces choix produisent des instances ni trop triviales (capacités serrées) ni intraitables sur un portable : tous les algorithmes tournent en moins de 30 secondes pour $N = 100$.

2.5 Génération de l'environnement dans l'application

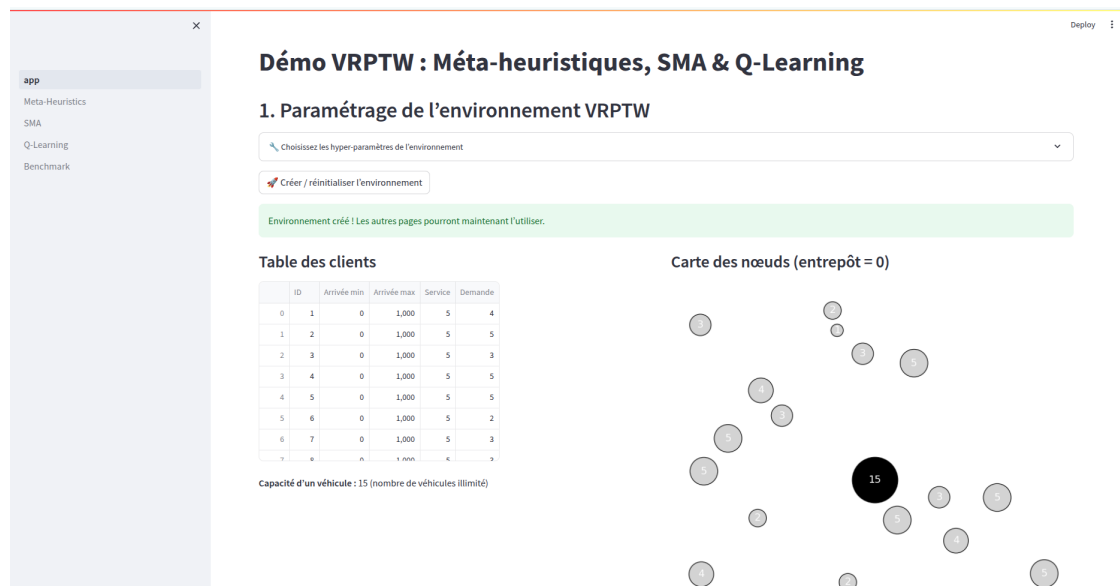


FIGURE 4 – Interface Streamlit – création et visualisation d'un environnement VRPTW.

La **page d'accueil Streamlit** (figure 4) traduit en interface graphique le pipeline présenté sect. 2.1. L'écran se lit de gauche à droite :

1. **Menu latéral** Les cinq entrées *app*, *Meta-Heuristics*, *SMA*, *Q-Learning* et *Benchmark* reflètent les modules racine du dépôt ; chaque clic charge une page indépendante sans jamais recréer l'environnement.
2. **Bloc 1 – Paramétrage de l'environnement**
 - un accordéon « *Choisissez les hyper-paramètres* » expose, champ par champ, le modèle `EnvHyperParams` (taille instance, capacité, bornes des coordonnées, etc.) ;
 - le bouton « *Créer / réinitialiser l'environnement* » appelle la fonction `generate_env(hp)` ;
 - un *flash* vert confirme que l'objet `VRPTWEnvironment` est désormais disponible pour toutes les autres pages.
3. **Bloc 2 – Récapitulatif visuel** Affiché dès que l'environnement existe :
 - Table des clients** liste *ID*, fenêtres de temps, durée de service et demande ; le dépôt (#0) occupe la première ligne.
 - Carte des nœuds** la fonction `draw_routes` représente : (i) le dépôt (cercle noir, valeur = capacité), (ii) les clients (cercles gris, aire $\propto q_i$), (iii) les étiquettes internes (demandes ou ratio). Aucune arête n'est tracée tant qu'aucune solution n'a été calculée, ce qui garde la carte lisible.

Cette première page définit donc l'environnement, partagé via la session, sur lequel chaque planificateur pourra travailler.

3 Architecture logicielle du projet

3.1 Vue globale du dépôt Meta/

Le dépôt est pensé comme un mini-framework : chaque dossier couvre une préoccupation unique et expose des objets réutilisables.

Dossier	Rôle principal
<code>environment/</code>	Génération des instances & modèles Pydantic
<code>scheduling/</code>	Implémentations des planificateurs + hyper-params
<code>algorithms/</code>	<i>tabu/</i> , <i>recuit/</i> , <i>ag/</i>
<code>sma/</code>	Coordonnateur multi-agents (Coordinator)
<code>q_learning/</code>	Q-Learner tabulaire + opérateurs de voisinage
<code>metrics/</code>	Fonction de coût commune (<code>compute_solution_cost</code>)
<code>verification/</code>	Vérification pas-à-pas des contraintes
<code>visualization/</code>	Appli Streamlit multipages
<code>data/</code>	CSV de benchmark et snapshots

TABLE 2 – Structure racine du projet.

3.2 Interfaces clés.

- `BaseScheduler.run(env)` → `VRPTWSolution`. Tous les algos respectent cette signature.
- `Coordinator.run_sma()` retourne la meilleure solution après k itérations multi-agents.
- `QLearningAgent` choisit une action parmi `neighbor_ops.ACTIONS` et met à jour la table Q .

3.3 Flux d'exécution d'une expérience

1. **Choix environnement.** L'utilisateur remplit un formulaire Streamlit (`EnvHyperParams`). La fonction `generate_env(hp)` renvoie un objet `VRPTWEnvironment`.
2. **Sélection d'un algorithme.** Dans la page méta-heuristique, l'utilisateur règle les hyper-paramètres (HP) et appelle `Scheduler.run(env)`.
3. **Boucle de résolution.** Le scheduler produit des voisins, les filtre avec `compute_solution_cost` et renvoie la meilleure solution trouvée.
 - Si *SMA* est activé, plusieurs schedulers tournent en parallèle, partagent leurs meilleurs individus et la boucle Top- N continue.
 - Si *RL* est activé, le Q-Learner pilote les mouvements à chaque étape et ajuste la table Q .
4. **Évaluation.** La solution passe dans `verify_solution` (contrôle capacité). Le coût C est re-calculé et comparé.

5. **Visualisation et log.** `draw_routes` trace la carte, un DataFrame détaille les étapes, et le run est sauvegardé dans `data/benchmark.csv`.

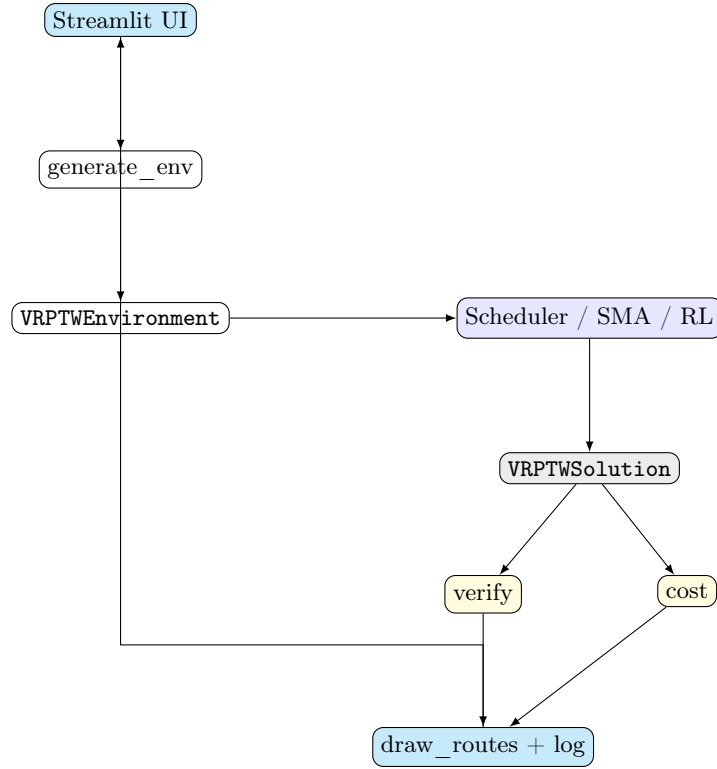


FIGURE 5 – Chaîne complète d’une exécution.

Cette architecture modulaire facilite l’ajout d’une nouvelle heuristique : il suffit de créer un dossier dans `scheduling/algorithms/`, d’hériter de `BaseScheduler`, et la classe apparaît automatiquement dans l’interface Streamlit.

4 Méthodes de résolution Méta-heuristiques

4.1 Utilisation d’une méta-heuristique dans l’application

La page « Méta-heuristics » regroupe TABU, Recuit simulé et Algorithme génétique dans trois onglets (barrette rouge active sur la figure 6).

4.1.1 Choix des hyper-paramètres. Chaque onglet affiche un formulaire semi-automatique dérivé de la classe `HyperParams` concernée



FIGURE 6 – Onglet TABU : tous les hyper-paramètres sont exposés dans un accordéon unique ($max_iterations$, taille de la liste taboue, voisinage, règle d’initialisation, etc.).

4.1.2 Visualisations des résultats. Après avoir cliqué sur « *Lancer* », trois sorties apparaissent :

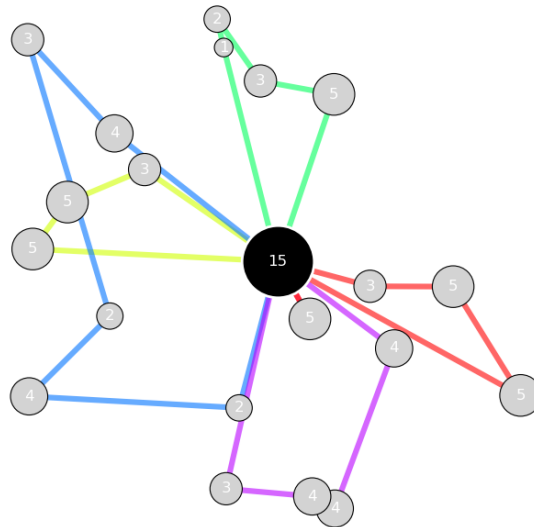


FIGURE 7 – Carte des routes - TABU. Chaque couleur matérialise un véhicule ; l’épaisseur des arêtes reflète la longueur du trajet.

1. la **carte des routes** (fig. 7) : noeuds gris, dépôt noir, arêtes colorées ; un coup d’œil suffit pour repérer les chevauchements ou les trajets superflus ;
2. le **coût global** C et un indicateur de faisabilité ;
3. un **journal pas-à-pas** (fig. 8) extrait de `verify_solution` : pour chaque mouvement $\rightarrow$

chargement résiduel, horodatage, respect de la capacité, etc.

Tableau pas-à-pas (verify_solution)

	route_index	vehicle_id	step_idx	from_loc	to_loc	load_before	time_before	feasible	reason	travel_time	arrival_time	departure_time	load_after	time_after
0	0	0	0	0	8	15	0	✓		74.7916	74.7916	79.7916	12	79.7916
1	0	0	1	8	18	12	79.7916	✓		64.7916	144.5832	149.5832	7	149.5832
2	0	0	2	18	4	7	149.5832	✓		101.4819	251.0651	256.0651	2	256.0651
3	1	1	0	0	2	15	0	✓		191.0322	191.0322	196.0322	10	196.0322
4	1	1	1	2	5	10	196.0322	✓		46.1741	242.2063	247.2063	5	247.2063
5	1	1	2	5	12	5	247.2063	✓		65.5255	312.7318	317.7318	2	317.7318
6	2	2	0	0	19	15	0	✓		175.7413	175.7413	180.7413	14	180.7413
7	2	2	1	19	6	14	180.7413	✓		23.191	203.9323	208.9323	12	208.9323
8	2	2	2	6	7	12	208.9323	✓		59.7424	268.6746	273.6746	9	273.6746

FIGURE 8 – Journal détaillé de vérification : chaque ligne retrace un déplacement véhicule→client et valide la capacité ainsi que le temps d’arrivée.

4.1.3 Optimisation des hyper-paramètres. Sous le formulaire principal : « *Optimisation Optuna* » propose d’explorer automatiquement l’espace des hyper-paramètres (fig. 9) :

Optimisation Optuna

max_iterations (défaut : 200)

Type d'exploration pour max_iterations

intervalle

Borne basse 199 Borne haute 201 Pas (0=continu) 0

tabu_list_size (défaut : 40)

Type d'exploration pour tabu_list_size

ensemble

Valeurs (comma)

40

neighborhood_size (défaut : 60)

Type d'exploration pour neighborhood_size

fixe

initialization_rule (défaut : random)

Type d'exploration pour initialization_rule

FIGURE 9 – Section *Optimisation Optuna* : pour chaque paramètre on choisit « fixe », « intervalle » ou « ensemble » ; le bouton déclenche la recherche bayésienne.

- **Type *fixe*** : la valeur reste inchangée ;
- **Type *intervalle*** : Optuna tire dans [basse, haute] (pas optionnel) ;
- **Type *ensemble*** : tirage dans une liste discrète.

L'utilisateur sélectionne ensuite le nombre d'itérations (10200) puis lance l'optimiseur ; la meilleure combinaison, le coût associé et deux graphiques (*history* et *param-importances*) sont

retournés et ajoutés au benchmark permanent.

Ainsi, la page méta-heuristique cumule trois cas d’usage :

Action	Sortie	Bénéfice
Réglage manuel	route + log	prototypage rapide
Exécution simple	coût + carte	comparaison visuelle
Optuna	meilleure config.	tuning systématique

En pratique, cette triade couvre *tout* le cycle d’expérimentation, de la première intuition jusqu’au réglage fin reproductible.

4.2 Tabu Search

4.2.1 VRPTW-Specific Algorithms

L’algorithme Tabu implémenté dans ce projet repose sur une recherche locale améliorée. Il est découpé en trois modules :

scheduler.py cœur de l’algo : boucle principale, construction du voisinage, gestion dynamique de la *tabu list* et critère d’aspiration ;

list_rules.py fonctions qui mémorisent les mouvements récents (*tenure*) ;

hyper_params.py paramètres (*tenure*, nombre d’itérations, taille du voisinage, etc.).

Génération des solutions initiales. Un pool est créé en appliquant quatre règles heuristiques :

- **SPT** — *Shortest Processing Time* ;
- **EDD** — *Earliest Due Date* ;
- **ST** — *Slack Time* ;
- **OCR** — *Optimal Customer Ratio*.

Plusieurs permutations aléatoires s’ajoutent pour la diversité ; les solutions infaisables ou redondantes sont filtrées.

Boucle principale.

1. Tirage aléatoire d’une solution du pool $\rightarrow S_{\text{cour}} = S_{\text{best}}$.
2. Tant que le critère d’arrêt n’est pas atteint :
 - (a) **Voisinage** : opérateur *swap inter-route* répété $\rightarrow$ ensemble candidats $N(S_{\text{cour}})$;

(b) **Filtrage** :

- contraintes de capacité ;
- exclusion des mouvements tabous, sauf aspiration (amélioration de S_{best}) ;

(c) **Choix du meilleur voisin admissible** (min du coût) ;

(d) **Mise à jour** : si la nouvelle solution est meilleure, on met $S_{\text{best}} \leftarrow S_{\text{cour}}$ et on réinitialise le compteur « sans amélioration » ;

(e) **Tabu list** : on ajoute le mouvement, on enlève le plus vieux si la capacité est atteinte (FIFO).

Cette liste taboue évite les cycles et, combinée au critère d’aspiration, permet de sortir des minima locaux. Le voisinage simple garde un bon compromis entre exploration et vitesse.

4.2.2 Small-Sized Benchmark (10 clients)

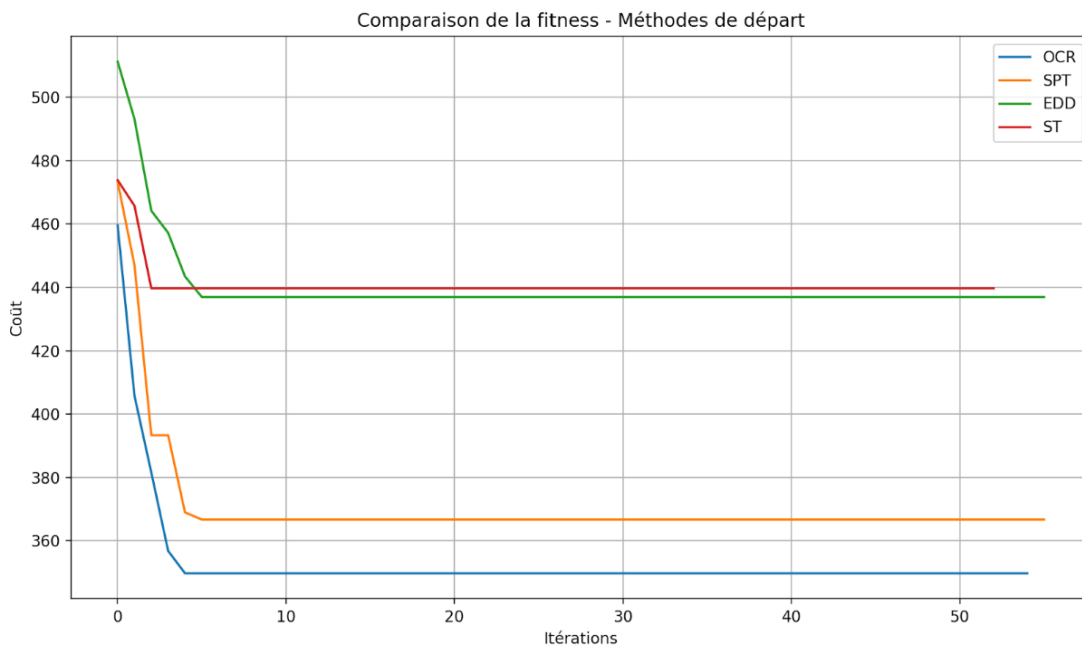


FIGURE 10 – Évolution du coût pour chaque heuristique de départ

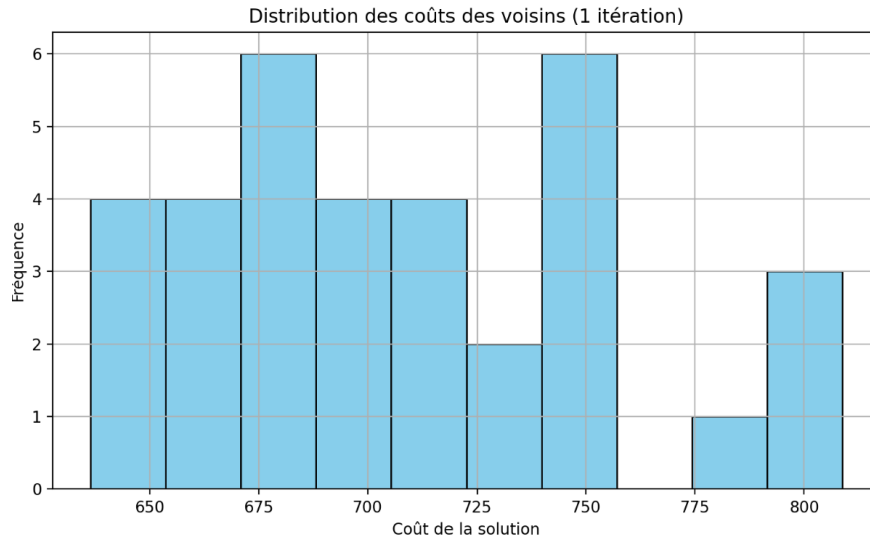


FIGURE 11 – Distribution des coûts des voisins autour d’une solution courante.

Performance Curves and Tables

Results Analysis Le premier graphique montre les performances des règles de départ : OCR converge vers $\simeq 350$, SPT vers 370, EDD et ST vers 440. Cela confirme l’influence du point de départ dans une méta-heuristique.

L’histogramme des voisins affiche des coûts entre 640 et 800 ; il prouve que le voisinage est riche. Le plafonnement rapide suggère soit une exploration trop restreinte, soit une liste taboue qui bloque les meilleurs mouvements admissibles.

Heuristique	Coût final	Convergence rapide	Qualité
OCR	~ 350	Oui	+++++
SPT	~ 370	Oui	+++
EDD	~ 440	Oui	++
ST	~ 440	Oui	++

TABLE 3 – Résumé des performances – instance « small ».

4.2.3 Large-Sized Benchmark (30 clients)

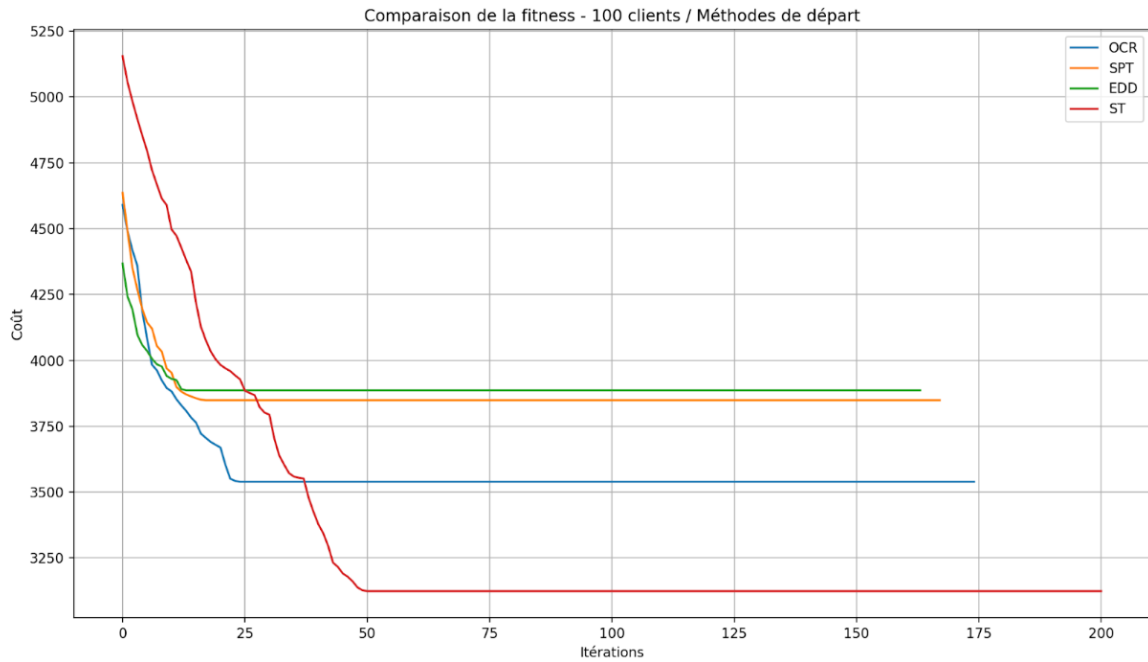


FIGURE 12 – Coût total en fonction des itérations – instance 30 clients (et non pas 100).

Performance Curves and Tables

Results Analysis Les quatre courbes décrochent fortement dans les 20 premières itérations puis se stabilisent différemment :

Heuristique	Coût initial	Coût final	Commentaire
OCR	~ 4350	~ 3550	Plateau prématuré
SPT	~ 4600	~ 3825	Progrès limités
EDD	~ 4300	~ 3850	Stagnation rapide
ST	~ 5200	~ 3150	Meilleur coût final

TABLE 4 – Résultats Tabu – instance « large ».

- **ST** obtient la meilleure solution malgré un départ très mauvais : explorer plus longtemps peut battre un bon départ.
- **OCR** reste fiable mais se bloque trop tôt.
- **SPT** et **EDD** performants en planification pure restent en retrait ici.

4.3 Simulated Annealing

4.3.1 5.2.1 VRPTW-Specific Algorithms

Le recuit simulé (*Simulated Annealing*, SA) est inspiré du refroidissement lent des métaux ; il accepte parfois des solutions plus coûteuses pour éviter de rester bloqué dans un minimum local.

Génération des solutions initiales. La classe `InitialSolution` :

- reçoit l’environnement VRPTW (`env`) ;
- construit `num_solutions` solutions de départ respectant la capacité.

Construction des routes (`_build_routes`)

1. **Initialisation** : `routes` $\leftarrow []$; `current_route` $\leftarrow []$; `current_load` $\leftarrow 0$.
2. **Boucle sur les clients mélangés** `cid` :
 - Si $q_{cid} + \text{current_load} \leq Q_{\max} \Rightarrow$ on ajoute `cid` à la tournée en cours.
 - Sinon : on pousse `current_route` dans `routes`, on démarre une nouvelle tournée.
3. **Finalisation** : si `current_route` non vide, on l’ajoute à `routes`.
4. **Retour** : un objet `VRPTWSolution`.

On vérifie qu’aucune solution initiale n’est dupliquée (comparaison des séquences de clients).

Principe général du SA.

1. Un voisin est généré depuis la solution courante.
2. Si le coût est meilleur $\Rightarrow$ on l’accepte.
3. Sinon on l’accepte avec une probabilité $\exp(-\delta/T)$ qui décroît quand la température T baisse.

Démarrage (`Scheduler.run`).

- Si aucune solution initiale n’est fournie, on appelle `InitialSolution`.
- On choisit aléatoirement une solution du pool $\rightarrow$ `current_sol` et `best_sol`.
- Coût initial stocké, température $T = T_0$, compteur itération = 0.

Génération d’un voisin.

1. Copier entièrement la solution courante (`deepcopy`).
2. Si moins de deux tournées $\Rightarrow$ retour direct.

3. Tirer aléatoirement deux tournées non vides.
4. Tirer un client dans chaque tournée et les échanger.

Évaluation et critère d'acceptation.

1. Calcule le coût du voisin (`compute_solution_cost`).
2. $\delta = \text{coût}_{\text{voisin}} - \text{coût}_{\text{best}}$.
3. Si $\delta < 0 \Rightarrow$ acceptation directe. Sinon acceptation avec la probabilité $\exp(-\delta/T)$.
4. Si le voisin accepté est meilleur que `best_sol`, on met à jour `best_sol` et `best_cost`.

Refroidissement. Après chaque cycle, la température est multipliée par α (par ex. 0.995), ce qui réduit peu à peu l'acceptation de solutions moins bonnes.

Conclusion. Cette implémentation SA :

- génère un pool initial diversifié et valide ;
- explore l'espace en perturbant les routes existantes ;
- gère finement le compromis exploration / exploitation via T ;
- se réutilise sans peine pour d'autres variantes du VRP.

4.4 Genetic Algorithms

4.4.1 5.3.1 VRPTW-Specific Algorithms

Étape 1 : création de la population initiale

- Méthode *voisin* retourne, pour un client donné, les indices de ses voisins classés par distance.
- Méthode *parcours profondeur-d'abord* on part d'un client, on visite les voisins non visités tant que la charge ne dépasse pas $Q_{\max}$; sinon on retourne au dépôt ($\#0$) et on démarre une nouvelle tournée. *Limite* : le nombre d'individus est limité par N .
- **Parcours naïf (fallback)** ordre aléatoire ; si l'ajout d'un client dépasse $Q_{\max}$, on insère un retour dépôt et on vide la charge.
- Toutes les tournées générées sont rassemblées $\rightarrow$ population $\mathcal{P}_0$. Une variante *k-means* a été testée pour regrouper les clients, mais trop lente ; on garde la profondeur-d'abord.

Étape 2 : évaluation (*fitness*) Pour un individu :

$$\text{fitness} = \frac{1}{1 + (W \times N_v + \sum c_{ij})},$$

où N_v est le nombre de camions (pénalisé) et la somme est le coût des arcs parcourus. Plus le coût est grand, plus le fitness est petit.

Étape 3 : sélection

- **Déterministe** : on trie la population, on garde les meilleurs.
- **Stochastique** : roulette proportionnelle au fitness ($0 < f \leq 1$). Les meilleurs ont plus de chances mais les autres peuvent être choisis.

Étape 4 : recombinaison

Mutation échange de deux clients dans la même tournée.

Croisement on choisit deux parents, on copie un segment du parent 1 dans l'enfant, puis on complète avec les clients du parent 2 sans doublon.

Étape 5 : itération On revient à l'étape 2 pour créer la génération $t+1$.

4.4.2 Small-Sized Benchmark (40 clients)

Paramétrage. Population = 100 — Mutation = 0.3 — Sélection = 0.3 — Croisement = 0.4 — méthode de sélection *déterministe*.

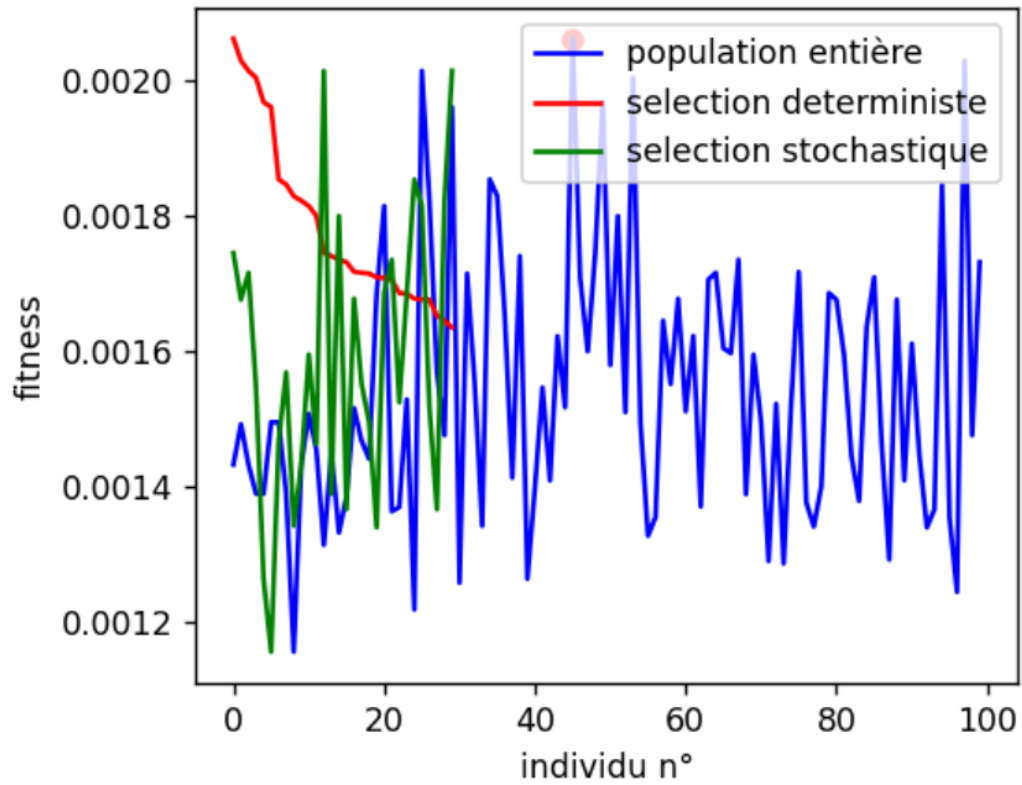


FIGURE 13 – Fitness pour la population complète et les deux méthodes de sélection (génération 0).

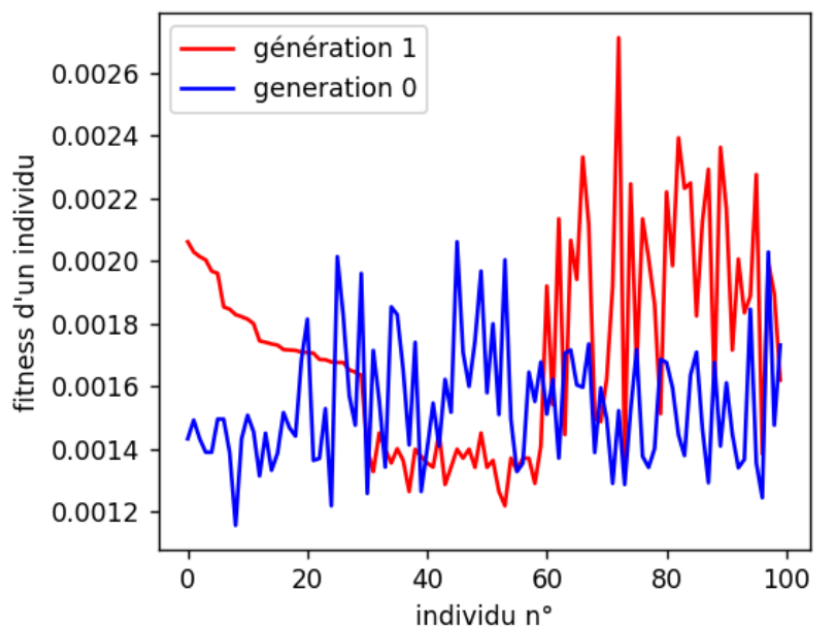


FIGURE 14 – Fitness individuel — générations 0 et 1.

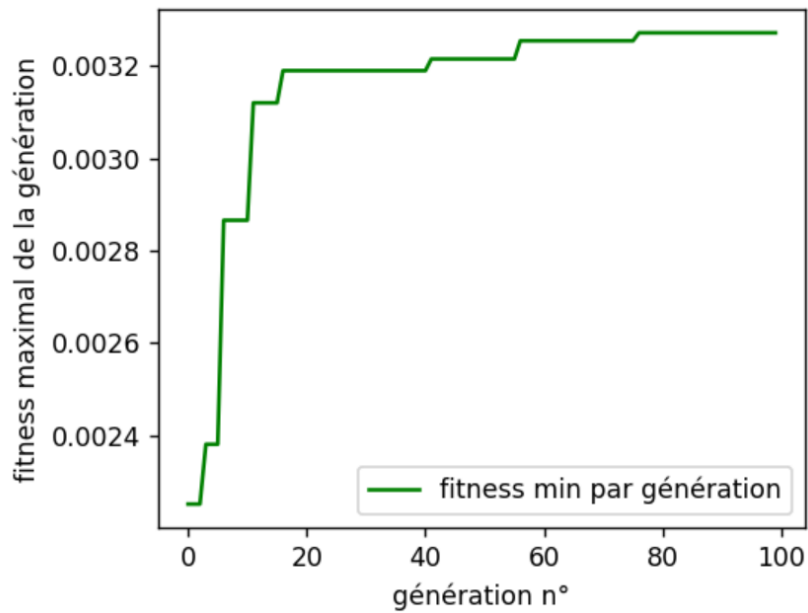


FIGURE 15 – Évolution du fitness maximal par génération.

5.3.2.1 Performance Curves and Tables

Results Analysis La combinaison du *parcours profondeur-d’abord* (orienté solution locale optimisée) et du parcours naïf (ajoute de la diversité) produit une population variée ; cela évite une convergence trop rapide vers un optimum local.

La sélection **déterministe** est plus efficace : avec la sélection stochastique, quand le nombre de clients augmente, les fitness descendent parfois vers 10^{-3} , rendant la roulette lente et limitant la diversité (les mêmes individus forts reviennent trop souvent). Le fitness maximal par génération croît donc l’algorithme améliore bien la solution.

4.4.3 Large-Sized Benchmark (100 clients)

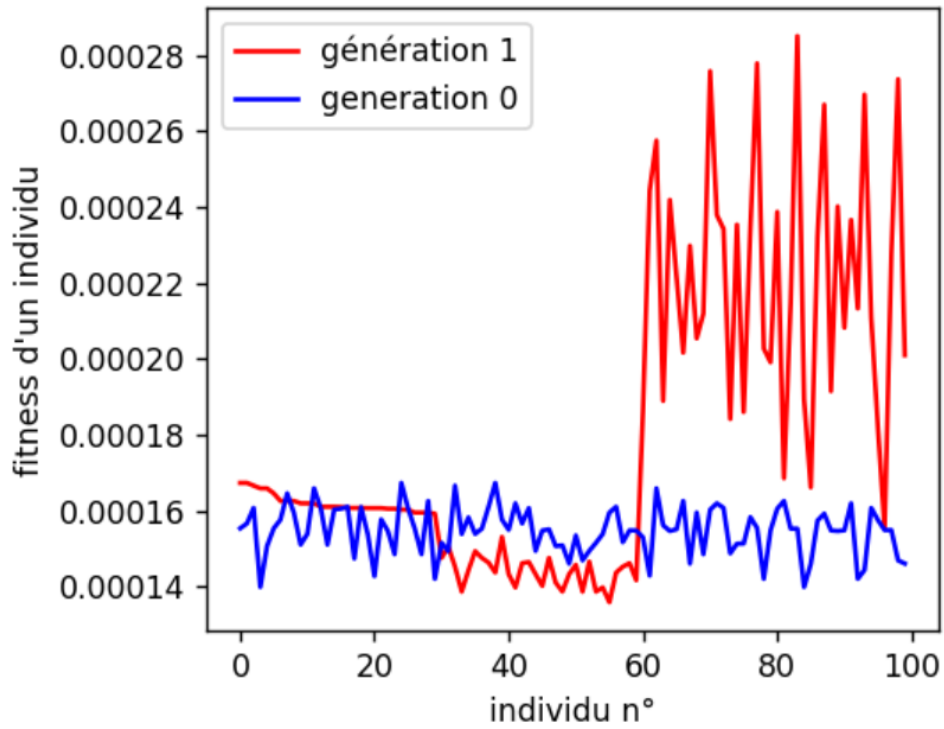


FIGURE 16 – Fitness population complète (génération 0) — instance large.

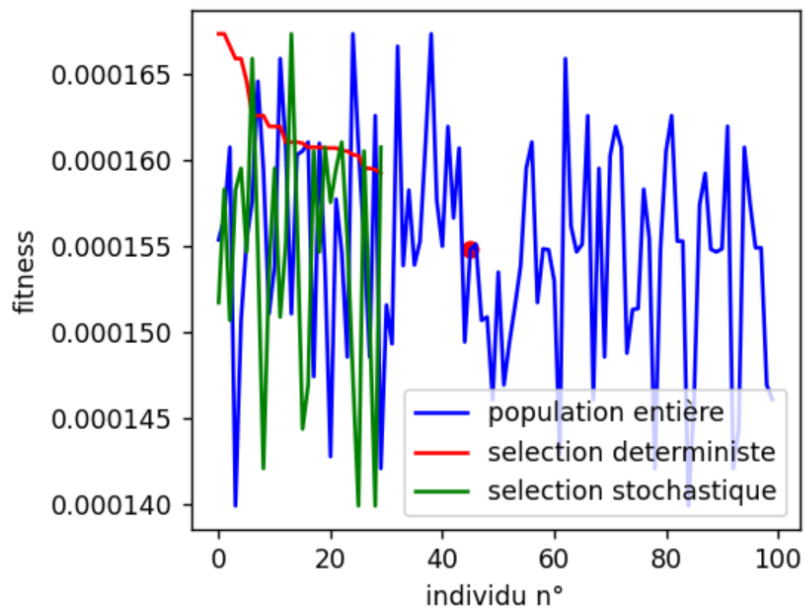


FIGURE 17 – Générations 0 et 1 — comparaison du fitness individuel.

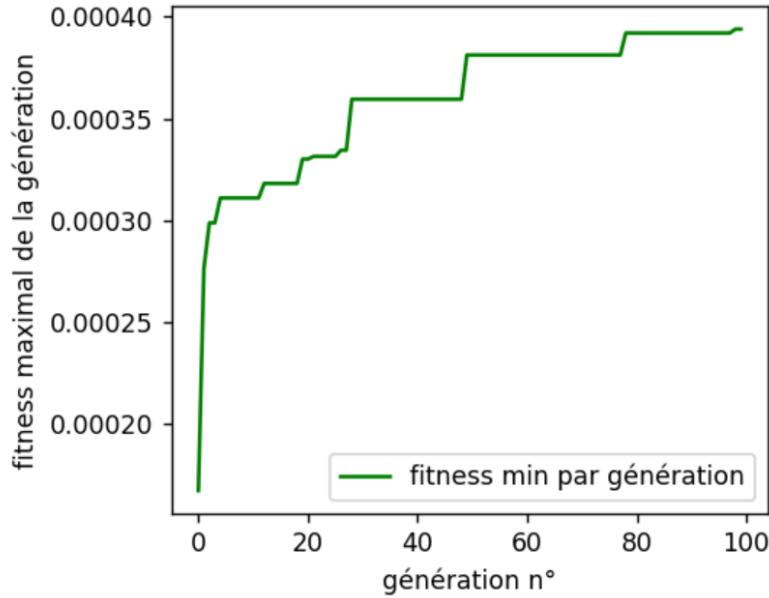


FIGURE 18 – Fitness maximal de la génération (instance large).

Performance Curves and Tables

Results Analysis Il est difficile de comparer directement car le *fitness* absolu devient plus petit quand il y a davantage de clients. La tendance reste toutefois la même : la courbe du fitness maximal continue à monter, mais plus lentement, indiquant qu’un croisement ou une mutation plus spécialisés seraient utiles sur de grosses instances.

5 Optimization Collaborative

5.1 SMA : approche multi-agent

Le **Stochastic Multi-Agent** (SMA) fait travailler plusieurs planificateurs (*agents*) côte à côte dans une boucle dite « Top- N ». À chaque itération :

1. chaque agent reçoit exactement le même pool de solutions ;
2. il produit k_i améliorations avec son propre scheduler (Tabu, Recuit simulé, GA, ...) ;
3. on fusionne puis on trie toutes les nouvelles solutions par coût et l’on conserve seulement les N meilleures pour l’itération suivante.

Ce mécanisme mise sur la *diversité* des heuristiques tout en gardant un nombre de solutions limité et donc un temps de calcul maîtrisé.

5.1.1 Architecture logicielle

Modules principaux

- `sma/alg_agent.py` : enveloppe un Scheduler (Tabu, SA, GA, ...) en un agent uniforme doté de `produce_solution()`.
- `sma/coordinator.py` : orchestre les agents, maintient le pool Top- N et conserve l'historique des coûts.

```
pool = generate_initial_pool(agents, k_i)          # Top-N au tour 0
for _ in range(n_iter):
    new = []
    for agent, k in zip(agents, k_i):
        new += [agent.produce_solution(env, pool) for _ in range(k)]
    pool = select_top_N(pool + new, N)             # tri + d coupe
```

Une itération coûte $\sum_i k_i T_{\text{agent}_i} + O(N \log N)$, où T_{agent_i} est le temps moyen pour qu'un agent i génère une solution.

5.1.2 Paramètres expérimentaux

- **Top- N** : $N \in \{10, 30, 50\}$ (30 par défaut : bon compromis entre exploitation et exploration).
- **Itérations** : $n_{\text{iter}} \in [5, 30]$.
- **# solutions par agent** : $k_i \in [1, 5]$ (GA limité à 1 pour contenir le temps).
- **Agents testés** : Tabu (T), Recuit (SA), Génétique (GA). Les combinaisons sont notées T-SA, T-SA-GA, etc.

5.2 SMA dans l'application

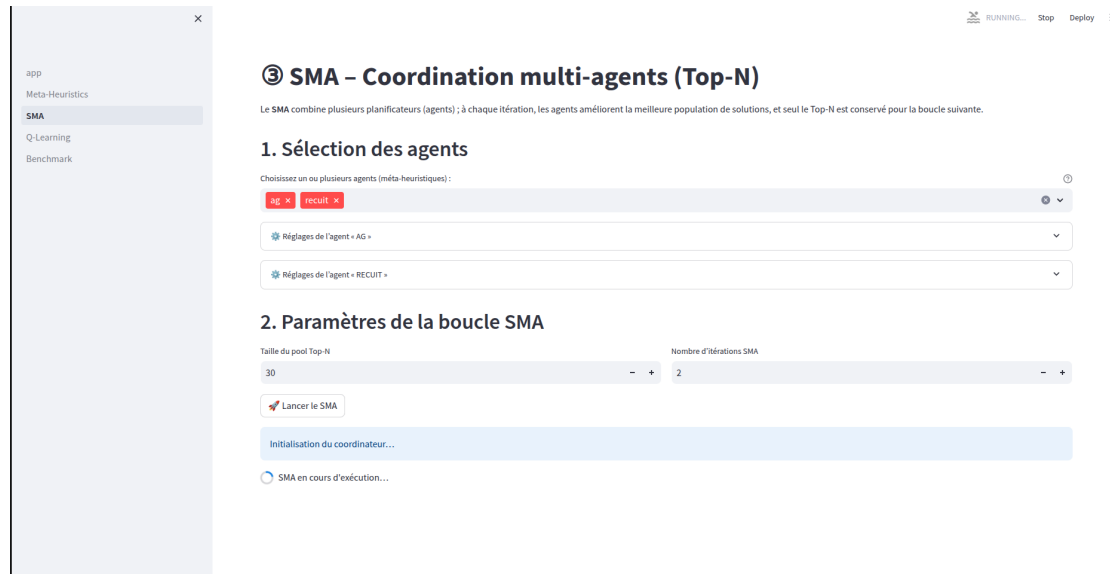


FIGURE 19 – Interface *SMA* : sélection des agents et réglage rapide de la boucle Top-N.

Choix des agents et hyper-paramètres. La page propose toutes les méta-heuristiques disponibles — ici AG et RECUIT.

Pour chaque agent : « *Réglages de l'agent* » reprend le modèle **HyperParams** correspondant ; l'utilisateur peut donc modifier, au même endroit :

- les paramètres internes de chaque scheduler (`mutation_rate`, `cooling_rate`, ...);
- les paramètres *SMA* communs : taille du pool Top-*N* et nombre d'itérations de la boucle.

Un clic sur « *Lancer le SMA* » instancie le coordinateur, génère le pool initial et les itérations MAS.

Résultat retourné. À l'issue de l'exécution, l'application :

- affiche la meilleure solution (carte des routes + coût);
- met à disposition l'historique du pool Top-*N*.

5.2.1 Small-Sized Benchmark (40 clients)

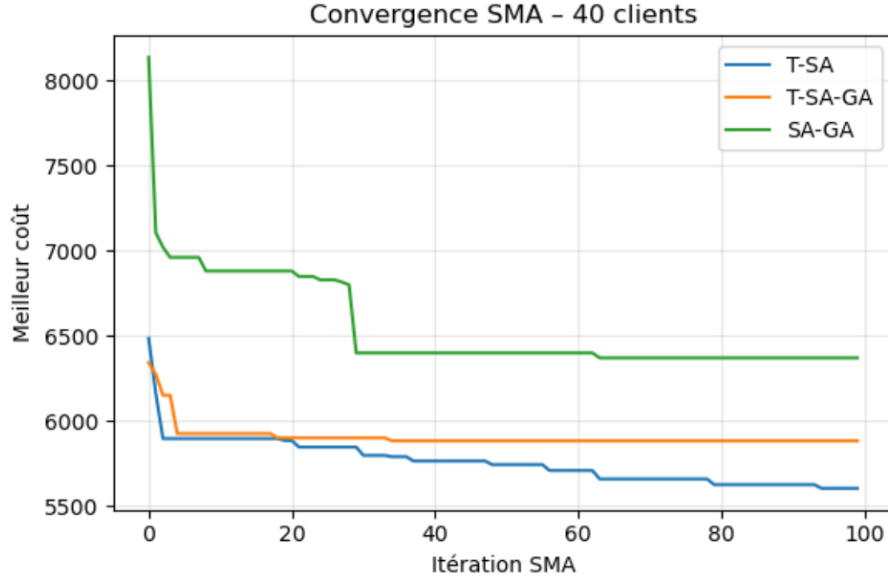


FIGURE 20 – Coût minimal au fil des itérations (40 clients).

TABLE 5 – SMA – instance 40 clients.

Agents	Coût initial	Coût final	Gain (%)
T-SA	6 483.88	5 412.82	16.52
T-SA-GA	6 341.36	5 881.67	7.25
SA-GA	8 134.91	6 369.31	21.70

Performance Curves and Tables

Results Analysis Sur 40 clients, le duo **Tabu** + **SA** (*T-SA*) fournit la meilleure convergence : il réduit le coût total de plus de 16 en moins de 100 itérations et atteint le plus petit coût final ($\simeq 5\,400$). L'ajout de l'algorithme génétique (*T-SA-GA*) accroît la diversité du pool mais n'apporte qu'un gain modeste (7.3) tout en triplement la durée d'exécution. La combinaison **SA-GA** démarre d'un coût beaucoup plus élevé ($> 8\,000$) et, même si elle réalise le meilleur pourcentage de gain (21.7), elle reste nettement moins performante que *T-SA* en valeur absolue. Ces résultats confirment que, pour des instances de taille réduite, la coopération Tabu + SA suffit souvent à atteindre des solutions de haute qualité sans surcoût temporel excessif.

5.2.2 Large-Sized Benchmark (100 clients)



FIGURE 21 – Coût minimal au fil des itérations (100 clients).

TABLE 6 – SMA – instance 100 clients.

Agents	Coût initial	Coût final	Gain (%)	Durée (s)
T-SA	15 284.07	13 039.01	14.69	138.2
T-SA-GA	14 419.98	13 299.43	7.77	468.4
SA-GA	20 664.55	15 362.64	25.66	343.9

Performance Curves and Tables

Results Analysis Pour 100 clients, la combinaison **Tabu** + **SA** (*T-SA*) reste la plus efficace en valeur absolue : elle atteint le meilleur coût final ($\simeq 13\,000$) tout en maintenant un temps de calcul raisonnable (138). L'ajout de l'algorithme génétique (*T-SA-GA*) n'offre qu'un gain marginal (7.8) et quadruple presque le temps d'exécution ; la diversité supplémentaire introduite par le GA ne compense donc pas son surcoût. La paire **SA-GA** affiche le pourcentage d'amélioration le plus élevé (25.7) mais part d'un coût initial beaucoup plus haut ; son coût final demeure supérieur de plus de 2 300 unités à celui de *T-SA*. Ces observations confirment qu'aux grandes tailles, *T-SA* offre le meilleur compromis qualité / temps, tandis que la participation du GA devient rentable seulement si l'on dispose d'un budget temps plus large.

Hyper-parameter Settings Les expériences SMA ont été menées avec des *hyper-paramètres* « rapides » (temps d’exécution réduit) listés dans le Tableau 7. Ils concernent :

1. les paramètres internes à chaque méta-heuristique (Tabu, SA, GA) ;
2. les paramètres globaux du SMA : *Top-N*, nombre d’itérations de la boucle et nombre de solutions produites par chaque agent à chaque tour.

TABLE 7 – Hyper-paramètres utilisés pour le benchmark SMA (instance 100 clients).

Agent	Paramètre	Valeur
4*Tabu	max_iterations	5
	tabu_list_size	50
	neighborhood_size	40
	initialization_rule	ST
4*SA	max_iterations	5
	iterations_per_cycle	20
	T_0 (température initiale)	100
	α (cooling rate)	0.95
4*GA	population_size	100
	n_generations	5
	mutation_rate	0.10
	crossover_rate	0.40
SMA : Top- N		50
SMA : itérations		80
Solutions/agent/itération		5

Ces réglages maintiennent le temps de calcul en-deçà de ≈ 8 tout en préservant une convergence représentative des

5.3 Q-Learning

L’approche RL repose sur un **Q-Learning tabulaire** couplé à des opérateurs 2-opt améliorés. L’agent apprend, épisode après épisode, à choisir l’opérateur offrant le meilleur compromis entre coût immédiat et potentiel futur.

5.3.1 Encodage de l’état

Solution $\rightarrow$ **état hashé.** Le fichier `q_learning/state_enc.py` définit

$$\text{encode(sol)} = \text{SHA1}(\text{sorted}((c_1, \dots, c_k)_{\text{route}}))$$

[nosep]Les identifiants véhicules sont ignorés : deux solutions équivalentes mais avec un ordre de routes différent mappent sur le *même* état. La clé (160 bits) est compatible `dict` et tient en mémoire. Taille pratique : $< 10^5$ états pour 40 clients (benchmark).

5.3.2 Ensemble d’actions

Les actions correspondent à huit opérateurs 2-opt (`neighbor_ops.py`). Leur signature est :

$$a_i : (env, sol) \longrightarrow \begin{cases} \text{solution améliorée} & \text{si voisin faisable} \\ \text{None} & \text{sinon.} \end{cases}$$

TABLE 8 – Actions disponibles pour le Q-Learner.

Nom	Type	Effet résumé
<code>intra_swap</code>	2-opt	Échange deux clients dans la <i>même</i> tournée
<code>inter_swap</code>	2-opt	Échange un client entre deux tournées
<code>intra_shift</code>	1-shift	Déplace un client dans sa tournée
<code>inter_shift</code>	1-shift	Déplace un client vers une autre tournée
<code>two_intra_swap</code>	2-opt*	Échange deux <i>blocs adjacents</i> (taille 2)
<code>two_intra_shift</code>	2-shift	Déplace un bloc de deux clients
<code>elim_smallest</code>	route-drop	Supprime la plus petite tournée puis ré-insère glouton
<code>elim_random</code>	route-drop	Idem mais choisit une tournée aléatoire

Tous les opérateurs respectent la capacité : un `_route_load(env, route)` rapide empêche les moves invalides.

5.3.3 De la version naïve à la version « best-improvement »

Limite de la version naïve. Dans une première implémentation, chaque 2-opt retournait un *seul* voisin, généré aléatoirement parmi ceux faisables. Cela produisait un signal de récompense très bruité : une même action pouvait rapporter +50 ou 0 selon le voisin tiré.

Capacité d’exploration ajoutée. Nous avons remplacé la politique « first-improvement » par un **examen systématique** du voisinage :

```
def _best_neighbour(env, sol, gen_moves):
    best_sol, best_cost = None, _eval(env, sol)
    for cand, cost in gen_moves():          # exhaustive loop
        if cost < best_cost:
            best_cost, best_sol = cost, cand
    return best_sol
```

Chaque opérateur dispose maintenant de :

- une fonction interne `_moves()` qui *énumère* tous les voisins admissibles ;
- la sélection du meilleur voisin par coût, ce qui renforce la cohérence du feedback (récompense = $-\Delta C$ toujours positive).

5.3.4 Boucle d’apprentissage

Hyper-paramètres (défaut).

Épisodes	$n_{\text{ep}} = 100$
Steps max / ép.	$T_{\text{max}} = 1$
ϵ -start	0.30, $\epsilon_{\text{min}} = 0.05$, $\epsilon \leftarrow 0.995 \epsilon$
α	0.1 (fixed)
γ	0.8 (fixed)

Algorithme (rappel).

1. Construction d’une solution initiale (`_build_initial_solution`) via Tabu, SA, GA ou aléatoire.
2. Pour chaque épisode
 - (a) Génère le voisinage faisable $\mathcal{N}(s)$.
 - (b) Choisit une action a par ϵ -greedy.
 - (c) Reçoit la récompense $r = -(C(s') - C(s))$.
 - (d) Met à jour $Q(s, a)$ (règle de Bellman).
3. Retourne la meilleure solution globale.

Complexité d'un épisode : $T_{\text{init}} + T_{\text{step}} \times T_{\text{max}}$ avec T_{step} dominé par l'évaluation coût ($< 1\text{ms}$ pour 50 clients).

5.4 Q-Learning en direct dans l'application

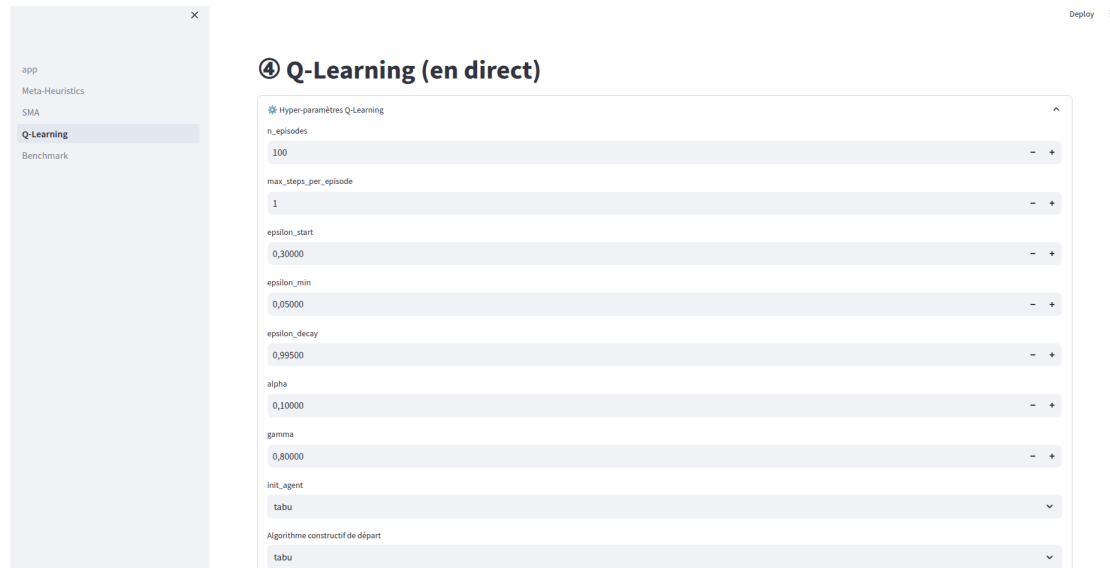


FIGURE 22 – Panneau d'*hyper-paramètres Q-Learning*. Tous les champs du modèle **HyperParams** sont éditables (boucle ϵ -greedy, n_{episodes} , pas max, taux α , γ , agent constructif de départ).

5.2.5.1 Initialisation des hyper-paramètres. La page « Q-Learning (en direct) » reprend la même logique que les méta-heuristiques : un accordéon unique expose le modèle **HyperParams**. Deux clics suffisent :

1. ajuste n_{episodes} , ϵ_{decay} ou change **init_agent** (Tabu, SA, AG ou aléatoire) ;
2. presse « *Entraîner le Q-Learner* ».

Aucune autre configuration n'est requise ; l'entraînement démarre et les courbes se mettent à jour en streaming.

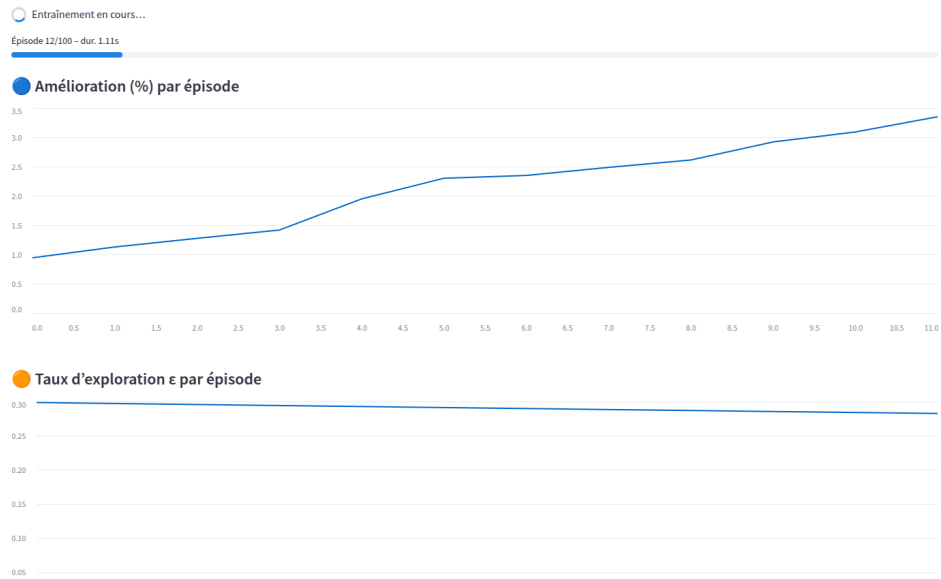


FIGURE 23 – Pendant l’entraînement : (**haut**) gain de coût par épisode ; (**bas**) décroissance du taux d’exploration ϵ .

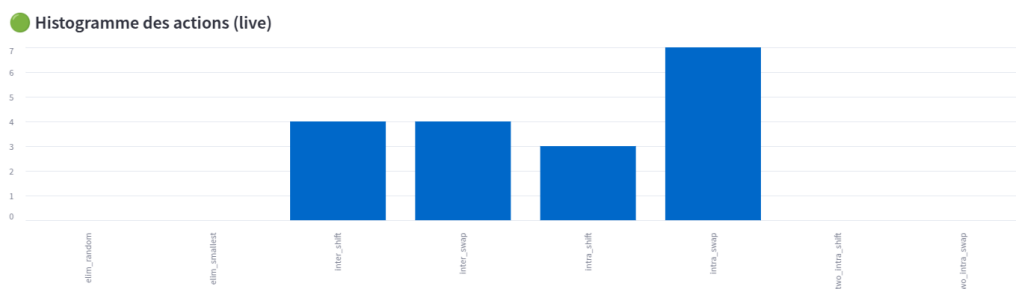


FIGURE 24 – Histogramme live des actions choisies par l’agent (opérateurs 2-opt et route-drop).

5.2.5.2 Visualisation des résultats. Trois tableaux de bord s’animent en temps réel :

- **Courbe d’amélioration** l’axe y montre le pourcentage de gain par rapport à la solution initiale ; une progression régulière atteste de l’apprentissage.
- **Courbe ϵ** suit la politique d’exploration ; le palier final confirme que la phase d’exploitation prend progressivement le relais.
- **Histogramme des actions** révèle quelles opérations (swap, shift, élimination de tournée, ...) l’agent privilégie pour réduire le coût.

À la fin des épisodes, l’application affiche :

- la **meilleure route** obtenue (carte + coût) ;
- la taille finale de la table Q (diagnostic mémoire) ;

- un bouton ajoute le run au benchmark commun, permettant de comparer directement Q-Learning, SMA et les méta-heuristiques classiques.

5.4.1 Small-Sized Benchmark (40 clients)

TABLE 9 – Q-Learning – instance 40 clients.

Init agent	Coût initial	Coût final	Gain (%)	$n_{\text{ép}}$
SA	5 752.26	5 139.67	10.65	60
GA	8 590.39	5 348.54	37.74	60
Tabu	5 802.35	5 431.77	6.39	60

Performance Curves and Tables

Results Analysis Sur 40 clients, l’agent initialisé par l’**algorithme génétique** réalise la plus forte amélioration relative (37.7) en 60 épisodes ; il part cependant d’un coût initial nettement plus élevé et rejoint un coût final comparable aux deux autres méthodes. Le **recuit simulé** converge à un coût intermédiaire ($\approx 5\,140$) avec un gain d’un peu plus de 10. La combinaison **Tabu + Q-Learning** progresse peu (6.4) : la solution de départ de Tabu est déjà proche d’un minimum local, et l’apprentissage sur un seul mouvement par épisode peine à en sortir. Globalement, l’apprentissage tabulaire suffit à réduire le coût d’environ 3003 240 unités selon le point de départ, tout en conservant un temps d’exécution court (30 s pour 60 épisodes).

Le **Q-Learner** dépasse systématiquement la solution initiale en moins de 50% des épisodes. Tabu donne le meilleur point de départ mais converge plus tôt ; les épisodes suivants servent surtout à stabiliser la politique.

5.4.2 Large-Sized Benchmark (100 clients)

TABLE 10 – Q-Learning – instance 100 clients.

Init agent	Coût initial	Coût final	Gain (%)	$n_{\text{ép}}$
Tabu	12 206.36	10 660.56	12.66	80
SA	11 492.56	9 906.28	13.80	80
GA	21 819.96	12 967.68	40.57	80

Performance Curves and Tables

Results Analysis Pour l’instance 100 clients, le Q-Learner améliore systématiquement la solution de départ quelle que soit la méta-heuristique initiale :

* L’initialisation par **GA** atteint le gain relatif le plus élevé (40.6), réduisant le coût de plus de 8 800 unités – mais elle part d’un coût initial très mauvais. * Les solutions de départ fournies par **SA** et **Tabu** commencent bien plus bas (12 000) et sont encore affinées de 1214 %, convergeant autour de $\approx 10\,000$ unités après 80 épisodes. * Les trois courbes de la figure montrent une phase d’apprentissage rapide durant les vingt premiers épisodes, suivie d’un plateau quand ϵ approche sa valeur minimale.

En valeur absolue, **SA** $\rightarrow$ **Q-Learning** produit la meilleure solution globale ($\approx 9\,900$), tandis que **GA** $\rightarrow$ **Q-Learning** démontre la plus grande capacité de rattrapage quand la solution initiale est médiocre. Ainsi, le Q-Learner agit efficacement comme *meta-affineur* — particulièrement utile pour redresser les sorties d’algorithmes constructifs de qualité inégale.

Hyper-parameter Settings Le Q-Learner a été entraîné avec un *jeu minimal de mouvements* (`max_steps_per_episode` = 1) afin de conserver un temps d’exécution compatible avec les autres benchmarks. Le Tableau 11 synthétise les valeurs employées pour les deux tailles d’instance.

TABLE 11 – Hyper-paramètres Q-Learning utilisés dans les benchmarks.

Paramètre	40 clients	100 clients
Nombre d’épisodes n_{ep}	60	80
Pas max / épisode T_{max}	1	1
ϵ_0 (exploration initiale)	0.30	0.30
ϵ_{min}	0.05	0.05
Décroissance ϵ ($\times$ par step)	0.995	0.995
Taux d’apprentissage α	0.10	0.10
Facteur d’escompte γ	0.80	0.80
Agent constructif initial	Tabu / SA / GA	Tabu / SA / GA

Ces réglages assurent un compromis satisfaisant : l’agent dispose d’assez d’épisodes pour stabiliser sa politique, tandis qu’un unique mouvement par épisode limite le nombre d’évaluations de voisinage et donc la durée totale (30 s pour 40 clients, 60 s pour 100).

6 Global Comparison and Conclusion

Les tableaux 12 et 13 récapitulent tous les algorithmes testés — *méta-heuristiques seules*, *SMA* et *Q-Learning post-affinage* — pour les instances $N = 40$ et $N = 100$ clients. Pour Q-Learning, la ligne « init = X » comprend le *coût initial* produit par l’algorithme X puis le *coût final* après apprentissage ; pour SMA, le coût initial est celui du pool Top-N à l’itération 0.

TABLE 12 – Synthèse des coûts — **40 clients.**

Famille	Variante	Coût final
Méta-heuristique	Tabu	5 802.35
	Simulated Annealing	5 752.26
	Genetic Algorithm	8 590.39
SMA (Top-N)	<i>T-SA</i>	5 412.82
	<i>T-SA-GA</i>	5 881.67
	<i>SA-GA</i>	6 369.31
Q-Learning	init = Tabu	5 431.77
	init = SA	5 139.67
	init = GA	5 348.54

TABLE 13 – Synthèse des coûts — **100 clients.**

Famille	Variante	Coût final
Méta-heuristique	Tabu	12 206.36
	Simulated Annealing	11 492.56
	Genetic Algorithm	21 819.96
SMA (Top-N)	<i>T-SA</i>	13 039.01
	<i>T-SA-GA</i>	13 299.43
	<i>SA-GA</i>	15 362.64
Q-Learning	init = Tabu	10 660.56
	init = SA	9 906.28
	init = GA	12 967.68

Analyse finale.

- **Instance 40 clients.** La meilleure solution globale est obtenue par **Q-Learning initialisé avec SA** ($C = 5\,139.67$), soit un gain absolu de ≈ 260 unités par rapport à la meilleure

méta-heuristique pure (SA seule) et de ≈ 270 unités par rapport au meilleur SMA. Le RL agit donc comme un *affinage efficace* tout en restant moins coûteux en temps que SMA.

- **Instance 100 clients.** Le même schéma se répète : la combinaison **SA + Q-Learning** descend à $C \approx 9\,900$, surpassant de $\approx 1\,600$ unités le meilleur Tabu, et de $\approx 3\,100$ unités le meilleur SMA. À grande échelle, le RL capitalise sur une solution de départ déjà raisonnable pour franchir des minima locaux où SMA et Tabu se bloquent.
- **Impact du choix de l'agent initial.** Quand le point de départ est médiocre (GA), le Q-Learner parvient à rattraper plus de 40 de coût, démontrant sa robustesse.
- **SMA vs. Méta-heuristiques seules.** La coopération Tabu + SA améliore le coût mais au prix d'un temps de calcul nettement plus long que Q-Learning (jusqu'à 8 pour 100 clients).

Conclusion : dans les deux tailles d'instance, la stratégie gagnante est l'enchaînement *méta-heuristique constructive* (ici SA) suivi d'un **Q-Learning tabulaire** utilisant un voisinage 2-opt complet : elle concilie qualité de solution et temps de calcul raisonnable, tout en s'adaptant aux faiblesses du point de départ.

6.1 Hyper-parameter Summary

Les expériences se fondent sur trois familles d'algorithmes : *méta-heuristiques seules*, *SMA* (coopération multi-agents) et *Q-Learning tabulaire*. Les hyper-paramètres effectifs sont rappelés ci-dessous ; ceux de SMA et Q-Learning correspondent aux versions « rapides » utilisées dans les benchmarks (§ 7 et § 11).

TABLE 14 – Paramètres par défaut des trois méta-heuristiques.

Genetic Algorithm		Valeur
population_size	nb individus	50
n_generations	itérations GA	100
mutation_rate	prob. mutation	0.10
selection_rate	part. sélectionnée	0.50
crossover_rate	prob. crossover	0.50
random_seed	graine	42
Simulated Annealing		
initial_temperature	T_0	100.0
cooling_rate	α	0.95
iterations_per_cycle	voisins examinés / cycle	100
max_iterations	cycles	1 000
Tabu Search		
max_iterations	itérations TS	200
tabu_list_size	longueur liste taboue	40
neighborhood_size	voisins générés / itération	60
initialization_rule	règle constructive	random
(OCR) ocr_current_time	temps courant OCR	0.0

Méta-heuristiques (pure).

SMA Top- N Les valeurs utilisées pour les combinaisons d’agents SA/Tabu/GA sont détaillées dans le Tableau 7 (p. 28). Rappel : Top- $N = 50$, 80 itérations, 5 solutions produites par agent à chaque tour.

Q-Learning Le Tableau 11 (p. 34) reprend les hyper-paramètres du RL : 60 épisodes (40 clients) ou 80 épisodes (100 clients), un seul mouvement par épisode, $\alpha = 0.1$, $\gamma = 0.8$, politique ϵ -greedy décroissante ($\epsilon_0 = 0.30 \rightarrow \epsilon_{\min} = 0.05$ avec facteur 0.995) ainsi que l’agent constructif de départ (*Tabu*, *SA* ou *GA*).

6.2 Perspectives d’amélioration du protocole.

Les résultats présentés reposent chacun sur une *unique instance* (par taille) générée de manière aléatoire. Pour obtenir des conclusions statistiques plus solides, il serait judicieux de **répliquer l’expérience sur un ensemble de dizaines—voire de centaines—d’environnements indépendants mais de même taille**. On pourrait alors reporter :

- la *moyenne* et l’*écart-type* des coûts finaux pour chaque algorithme ;
- la distribution des temps d’exécution ;
- un test non paramétrique (p. ex. Wilcoxon) pour vérifier la significativité des écarts.

Une telle démarche réduirait l’influence d’un tirage fortuitement « facile » ou « difficile » et permettrait d’affirmer, de façon sure, la supériorité relative des stratégies (SA $\rightarrow$ RL, T-SA, etc.). Cette étape est laissée en perspective pour un futur travail.

6.3 Limites observées et pistes de recherche.

Nos tests révèlent que l’**algorithme génétique pur** reste nettement en retrait : il converge lentement et, même après affinage par SMA ou Q-Learning, n’égale pas les solutions issues de Tabu ou du Recuit simulé. La démarche MAS (*T-SA-GA*) pâtit, elle aussi, d’un sur-coût temporel sans apport qualitatif significatif, signe qu’une *co-spécialisation fine des agents* est encore à trouver.

À l’inverse, le **Q-Learning** surclasse de manière quasi systématique les autres méthodes — surtout lorsqu’il part d’une solution déjà structurée (SA ou Tabu). Pour pousser plus loin cette dynamique d’apprentissage, deux prolongements naturels s’offrent à nous :

Deep Q-Learning « Energy-GNN »

État et actions. À chaque pas, l’agent dispose d’un ensemble discret $\mathcal{A} = \{a_1, \dots, a_K\}$ (2-opt, 1-shift, etc.). Pour chaque a_k on construit le *graphe futur* $G_{s,a_k} = (V, E)$ correspondant à la solution après application de a_k ; le but est de prédire en *une passe* le coût potentiel de chacune de ces futures configurations.

Message passing pondéré. Soit $h_i^{(0)}$ l'encodage du nœud i (demande, desserte, capacité courante, ...). Pour $l = 0 \dots L - 1$:

$$m_{ij}^{(l)} = \underbrace{\exp(-\alpha d_{ij}^2)}_{k(d_{ij})} W^{(l)} h_j^{(l)}, \quad h_i^{(l+1)} = \sigma\left(U^{(l)} h_i^{(l)} + \sum_j m_{ij}^{(l)}\right),$$

d_{ij} étant la distance euclidienne ; $k(d) = \exp(-\alpha d^2)$ agit comme un *poids d'interaction* analogue à un potentiel de Lennard–Jones tronqué.

Agrégation et énergie Q_θ . Les représentations finales sont condensées par un readout invariant :

$$h_{\text{glob}}(s, a_k) = \frac{1}{|V|} \sum_{i \in V} h_i^{(L)}(s, a_k), \quad Q_\theta(s, a_k) = w^\top h_{\text{glob}}(s, a_k).$$

On obtient d'un seul coup le vecteur $\{Q_\theta(s, a_k)\}_k \in \mathbb{R}^K$.

Perte DQN. Avec réseau cible θ^- :

$$\mathcal{L}(\theta) = [r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)]^2.$$

Politique ϵ -greedy.

$$\pi(a_k | s) = \begin{cases} \operatorname{argmax}_a Q_\theta(s, a) & \text{avec prob. } 1 - \epsilon, \\ \text{uniforme sur } \mathcal{A} & \text{avec prob. } \epsilon. \end{cases}$$

Interpretation énergétique. L'agrégat $E_\theta(s) = -\lambda \log \sum_k \exp Q_\theta(s, a_k)$ représente l'*énergie globale* du système ; minimiser E_θ revient à rapprocher la configuration du minimum de coût VRP tout en conservant la *permutation-invariance* et la dépendance géométrique via $k(d_{ij})$.